

UNCLE TED SYSTEM DEFENSE SUITE

Architectural Specification, Cryptographic Engineering, and Complete Systems Deployment Manual

A Cabin in the Digital Woods

EXHAUSTIVE PLATFORM REFERENCE AND RUNTIME SPECIFICATION

Platform Targets: Android 9.0 to 16.0 (API Levels 28-34+)

Module Architecture: ARMv8.5-A / ARM64-v8a Native Execution Layer

Release Tag: v8.0.1 · Production Hardening Edition

Lead Architect & Systems Engineer:

Hamoon Soleimani

<https://hamoon.net/> · GitHub: [HamoonSoleimani](#)

Licensed under the Open MIT License · Engineered for Defensive Auditing and Anti-Seizure Protection

Document Generated: September 2026 · Revision Index: 8.0.1-EXT

Notice of Irreversible Operation & Legal Disclaimer

CRITICAL OPERATIONAL WARNING

READ THIS NOTICE CAREFULLY PRIOR TO PROVISIONING, COMPILING, OR FLASHING THIS CODEBASE.

The operational mechanisms, protocols, and native drivers detailed in this technical specification interface directly with low-level Linux kernel abstractions, physical Universal Flash Storage (UFS) controllers, Power Management ICs (PMIC), cellular baseband modems, and discrete hardware security coprocessors (including Google Titan M/M2 and Qualcomm Secure Processing Units).

When armed, subsystems such as:

1. **The Fast Crypto-Shred Engine** (`FastCryptoShredEngine.kt`),
2. **The JEDEC Silicon Discard Native Bridge** (`native-security.cpp`),
3. **The Titan M2 Silicon Suicide Register** (`StrongBoxSecurityManager.kt`),
4. **The Physical USB PHY Annihilation Bus** (`UsbTrapdoorController.kt`), and
5. **The Vold Kernel Keyring Eviction Loop** (`EphemeralKeyDecayEngine.kt`)

execute **irreversible, unrecoverable, silicon-level cryptographic erasure**.

Executing a cryptographic suicide sequence revokes the master Key Encryption Keys (KEKs) sealed in hardware silicon, overwrites File-Based Encryption (FBE) metadata directories, issues synchronous `BLKSECDISCARD` and `BLKDISCARD` ioctl commands directly to raw block storage, and stages low-level Bootloader Control Block (BCB) format flags. **Under no circumstances can data sanitized through these mechanisms be restored, recovered, reconstructed, or salvaged by any forensic laboratory, mathematical cryptanalysis, or chip-off physical extraction workstation.**

This documentation and its underlying codebase are engineered strictly for academic systems research, authorized defensive security auditing, and human rights defense against unlawful seizure and physical coercion. **Never deploy this software to any device without explicit, written, and informed authorization from the device owner.** The authors, contributors, and maintainers assume no liability for catastrophic data loss, silicon invalidation, or hardware failure resulting from the operation of this architecture.

Document Executive Summary

This manual provides an exhaustive, unabridged reference for the **Uncle Ted System Defense Suite (v8.0.1)**. It bridges low-level hardware silicon physics with user-facing anti-coercion interfaces on modern Android operating systems (Android 9.0 through 16.0+).

Unlike commercial security tools that operate purely within unprivileged userspace, Uncle Ted operates across two mutually exclusive privileged profiles:

- **Route A (Enterprise Device Owner Mode):** Operates within an intact hardware chain of trust under a locked bootloader and enforcing Android Verified Boot (AVB 2.0). It leverages discrete Titan M2 KeyMint capabilities, hardware monotonic anti-rollback anchors, exponential Weaver rate-limiting, and native USB HAL differential signaling cutoffs.
- **Route B (Privileged Root and Hook Mode):** Operates under Magisk, KernelSU, KernelSU-Next, or APatch paired with LSPosed framework hooks in `system_server`. It implements in-process lockscreen credential interception, atomic sub-OS decoy user switching, direct Qualcomm DWC3 register unbinding, true 4KB File-Based Encryption key-shredding, and JEDEC silicon erase commands.

Every subsystem, cryptographic equation, native C++ routine, and deployment pipeline is detailed without summarization.

Contents

Notice of Irreversible Operation & Legal Disclaimer	i
Document Executive Summary	ii
I Foundations of Anti-Forensics and Threat Modeling	1
1 The Physical Seizure Threat Model	2
1.1 Taxonomy of Forensic Extractions	2
1.1.1 1. Commercial Forensic Workstations (Cellebrite UFED, Magnet GrayKey) .	2
1.1.2 2. NAND Mirroring and State Snapshot Rewinds	3
1.1.3 3. Emergency Download Mode (EDL) & BROM Exploitation	3
1.1.4 4. RF Downgrade Attacks (IMSI-Catchers / Stingrays)	3
1.1.5 5. Physical Coercion and Keyguard Compulsion	3
1.2 Temporal Windows of Vulnerability: AFU vs. BFU	3
2 Architecture Blueprints: Route A vs. Route B	5
2.1 The Platform Security Dilemma on Modern Android	5
2.2 Route A: Hardware-Backed Device Owner Mode	5
2.3 Route B: Privileged Root and Hook Mode	6
II Cryptographic Engineering and Quantum Resistance	7
3 Hardware KeyMint and Silicon Suicide	8
3.1 Discrete KeyMint Hardware Integration	8
3.2 Sub-10ms Silicon Suicide Mechanics	9
3.3 Monotonic Counter Synchronization and Anti-NAND Mirroring	9
4 Out-of-Band Key Decoupling via EC-OPRF (RFC 9497)	10
4.1 Mathematical Principles of Oblivious Pseudorandom Functions	10
4.1.1 1. Point Blinding Phase (Client)	10
4.1.2 2. Blind Evaluation Phase (Server)	11
4.1.3 3. Unblinding and Key Derivation Phase (Client)	11
4.2 Code Implementation (<code>OprfClientEngine.kt</code>)	11

5	Post-Quantum Hybrid Cryptography (FIPS 203)	13
5.1	Module Learning with Errors (ML-KEM-768)	13
5.2	Hybrid Combiner Implementation (PostQuantumEngine.kt)	13
III	Memory Hardening, Exploit Mitigation, and Key Decay	15
6	Volatile Memory Hardening and Exploit Defense	16
6.1	The Inverted Dead-Man Fail-Closed Engine	16
6.2	ARMv8.5-A Memory Tagging Extension (MTE)	16
6.2.1	Native C++ MTE Enforcement (native-security.cpp)	17
6.3	Native Memory Barriers and Memory Pinning	17
IV	Physical Bus Annihilation and Hardware Sentinels	19
7	Zero-Latency Hardware USB PHY Annihilation	20
7.1	The Race Condition of USB Forensic Enumeration	20
7.1.1	1. Route A (Device Owner USB HAL v1.3+)	20
7.1.2	2. Route B (Direct Kernel DWC3 / UDC Manipulation)	20
7.1.3	3. 100ms Fast-Path Intrusion Panic	21
8	Silicon Discard and Storage Sanitization	22
8.1	Wear-Leveling and the Failure of Traditional Overwrites	22
8.2	The True FBE 4KB Key-Shredding Pipeline	22
8.2.1	Native C++ Block Discard IOCTL Implementation	23
9	Advanced Hardware Environmental Sentinels	24
9.1	PMIC Battery Micro-Telemetry Sentinel	24
9.2	Modem 2G Masking and Stingray Defense	25
V	Plausible Deniability, Decoys, and Covert Traps	26
10	DNG Polyglot Vault and Sub-OS Decoys	27
10.1	Plausible Deniability DNG Polyglot Vault	27
10.2	Native Hooking & In-Process Decoy Space Migration	27
10.2.1	Salted Hash Bridge Architecture	28
10.2.2	In-Process Session Migration in system_server	28
VI	Covert Signaling and Out-of-Band Remote Control	29
11	Zero-Knowledge Signaling and Remote C2	30
11.1	Oblivious HTTP Covert Canaries (RFC 9458)	30
11.1.1	Google Telemetry Masquerade Format	30
11.2	Asymmetric Ed25519 Binary Wire Protocol	31
11.3	Single-Use Emergency Recovery Tokens (OTC)	31

VII Complete Deployment, Compilation, and Verification Manual	32
12 Compilation, Packaging, and Environment Setup	33
12.1 Development Environment Configuration	33
12.2 Building Release Artifacts	33
12.3 Universal Packager Architecture (<code>package_module.py</code>)	34
13 Deployment Guide: Route A and Route B	35
13.1 Route A Deployment: Hardware-Backed Device Owner	35
13.2 Route B Deployment: Root, LSPosed, and Systemless Priv-App	36
14 Verification, Diagnostics, and Operational Auditing	37
14.1 1. Confirming Strategy Resolution	37
14.2 2. Verifying Synchronous ARM MTE Memory Tagging	37
14.3 3. Auditing USB Line Severing on Screen Lock	37
14.4 4. Auditing LSPosed Keyguard Hook Interception	38
14.5 5. Validating Fail-Closed OPRF Derivation	38
14.6 6. Auditing Asymmetric Remote Wire Commands	38
References	39

Part I

Foundations of Anti-Forensics and Threat Modeling

Chapter 1

The Physical Seizure Threat Model

1.1 Taxonomy of Forensic Extractions

Traditional mobile security models rely on a perimeter defense architecture: the device is assumed to remain in the physical custody of its owner, shielded by lockscreen authentication barriers. This model breaks down when a device is physically confiscated by state actors, corporate intelligence teams, or forensic laboratories equipped with automated extraction workstations.

Forensic extraction methodologies fall into five distinct categories:

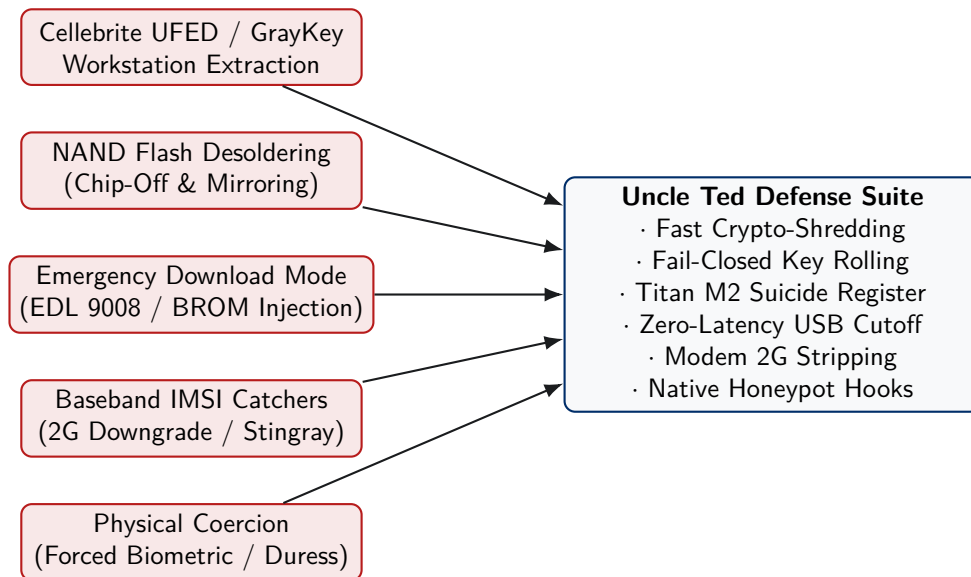


Figure 1.1: Adversarial Vectors Targeted by the Uncle Ted Suite

1.1.1 1. Commercial Forensic Workstations (Cellebrite UFED, Magnet GrayKey)

Forensic workstations interface with locked mobile devices over USB. Upon connection, the workstation interrogates the device's USB descriptor endpoints, attempts fastboot or ADB authentication, and leverages known zero-day vulnerabilities in the USB stack (e.g., Linux kernel UDC gadget drivers or vendor-specific USB PHY controllers like Qualcomm DWC3) to execute kernel memory dumps.

1.1.2 2. NAND Mirroring and State Snapshot Rewinds

NAND mirroring bypasses attempt counters enforced by operating system software. An attacker desolders the Universal Flash Storage (UFS) or eMMC silicon chip from the PCB, mounts it on a dedicated high-speed BGA reader, and creates a raw byte-for-byte image of the storage partitions. The chip is resoldered, brute-force PIN guesses are attempted until the device locks out, and the original snapshot is re-flashed. This resets the attempt counter on disk, allowing an infinite number of password guesses.

1.1.3 3. Emergency Download Mode (EDL) & BROM Exploitation

SoCs feature read-only primary bootloaders baked into mask ROM (e.g., Qualcomm EDL 9008, MediaTek BROM). By shorting physical test points on the motherboard or sending proprietary USB handshake sequences, examiners force the SoC into recovery modes that bypass Android Verified Boot entirely, dumping raw memory blocks directly over USB.

1.1.4 4. RF Downgrade Attacks (IMSI-Catchers / Stingrays)

Cell-site simulators impersonate legitimate carrier cell towers by broadcasting high-power signals. They take advantage of the fact that cellular modems prefer the strongest local signal. Because the GSM (2G) protocol standard lacks mutual authentication and encryption, the rogue transceiver forces the device down to 2G, capturing unencrypted voice and SMS data, and injecting malicious over-the-air binary configuration frames.

1.1.5 5. Physical Coercion and Keyguard Compulsion

Adversaries subject the operator to interrogation or physical threats to compel disclosure of the unlock credentials. Standard devices offer no middle ground: either the operator enters their real PIN, exposing all data, or they refuse, inviting further coercion.

1.2 Temporal Windows of Vulnerability: AFU vs. BFU

Android File-Based Encryption (FBE) creates two distinct cryptographic device states:

State	Cryptographic Reality	Vulnerability Surface
Before First Unlock (BFU)	The device has booted, but the operator has not authenticated. Credential-Encrypted (CE) keys exist exclusively as encrypted blobs inside the Secure Element. Only Device-Protected (DE) storage is readable.	Minimal. Offline attacks must crack the hardware KeyMint or exploit low-level bootloader/boot ROM vulnerabilities.
After First Unlock (AFU)	The operator has authenticated once. The CE root Key Encryption Keys (KEKs) have been decapsulated from the Secure Element and reside directly in volatile Linux kernel RAM within the <code>fsencrypt</code> keyring.	Extreme. The keys to the filesystem reside in physical RAM. DMA attacks, USB kernel exploits, or chassis unsealing with bench power supplies can extract all user data.

Table 1.1: Cryptographic Analysis of BFU vs. AFU Operational States

The primary goal of Uncle Ted is to make the AFU state as fleeting as possible: dropping the device back into cold BFU within milliseconds of detecting any anomaly, and shredding all keys if tampering is confirmed.

Chapter 2

Architecture Blueprints: Route A vs. Route B

2.1 The Platform Security Dilemma on Modern Android

On Android 9 through 16, security enforcement is tightly linked to the status of the bootloader:

ARCHITECTURAL INVARIANT

The Bootloader Integrity Trade-off:

- Locking the bootloader enforces **Android Verified Boot (AVB 2.0)**. Cryptographic public keys stored in read-only hardware registers verify the signature of every boot partition. Hardware attestation succeeds, and the discrete KeyMint (Titan M2) allows hardware-isolated key generation. However, userspace lacks root authority to hook system services or execute low-level block-discard commands.
- Unlocking the bootloader permits superuser execution (Magisk, KernelSU, APatch) and framework hooking (LSPosed), granting the ability to intercept calls inside `system_server` and command kernel block controllers. However, the hardware root of trust is severed: AVB enters the **orange** unverified state, and discrete hardware KeyStore attestation fails.

Uncle Ted resolves this dilemma by supporting two deployment paths:

2.2 Route A: Hardware-Backed Device Owner Mode

Route A is designed for operators facing physical forensic seizure who require the hardware root of trust to remain intact.

- **Bootloader State:** Unlocked (**orange** boot state) with custom kernel or root provider (Magisk, KernelSU, APatch) and LSPosed framework.
- **Privilege Model:** Superuser execution (UID 0 / root) paired with Xposed hooks injected into the `system_server` runtime.

- **Lockscreen Interception:** The native hook (`LockscreenHook.kt`) intercepts `verifyCredential()` calls before authentication is evaluated, identifying Duress, Wipe, or Honeygot PINs at input time.
- **Decoy Space Migration:** When the Honeygot PIN is entered, the hook commands `switchUser(10)` inside `system_server`, seamlessly transitioning to a secondary decoy profile (User 10) without exposing User 0 data.
- **Hardware Destruction:** Bypasses userspace filesystem abstractions to write directly to `/dev/block/by-name/metadata` and dispatch `BLKSECDISCARD` ioctl calls to the raw storage hardware.
- **USB Severing:** Unbinds the Qualcomm DWC3 driver registers and Linux USB Device Controller (UDC) endpoints via root shell commands.

2.3 Route B: Privileged Root and Hook Mode

Route B is designed for environments where the primary threat is physical coercion or live operational capture, requiring fine-grained control over the operating system.

- **Bootloader State:** Unlocked (orange boot state) with custom kernel or root provider (Magisk, KernelSU, KernelSU-Next, APatch) and LSposed framework.
- **Privilege Model:** Superuser execution (UID 0 / root) paired with Xposed hooks injected into the `system_server` runtime.
- **Lockscreen Interception:** The native hook (`LockscreenHook.kt`) intercepts `LockSettingsService.verifyCredential()` calls before authentication is evaluated, identifying Duress, Wipe, or Honeygot PINs at input time.
- **Decoy Space Migration:** When the Honeygot PIN is entered, the hook commands `IActivityManager.switchUser(10)` inside `system_server`, seamlessly transitioning to a secondary decoy profile (User 10) without exposing User 0 data.
- **Hardware Destruction:** Bypasses userspace filesystem abstractions to write directly to `/dev/block/by-name/metadata` and dispatch `BLKSECDISCARD` ioctl calls to the raw storage hardware.
- **USB Severing:** Unbinds the Qualcomm DWC3 driver registers and Linux USB Device Controller (UDC) endpoints via root shell commands.

Part II

Cryptographic Engineering and Quantum Resistance

Chapter 3

Hardware KeyMint and Silicon Suicide

3.1 Discrete KeyMint Hardware Integration

The Google Titan M2 is an independent, hardened microcontroller with dedicated RAM, flash memory, and an onboard cryptographic hardware accelerator, connected to the application processor over an internal SPI/I2C bus.

Uncle Ted interfaces with this hardware via `StrongBoxSecurityManager.kt`, which generates an AES-256-GCM master suicide key inside the discrete coprocessor:

```
1      val keyGenerator = KeyGenerator.getInstance(  
2      KeyProperties.KEY_ALGORITHM_AES,  
3      "AndroidKeyStore"  
4      )  
5      val specBuilder = KeyGenParameterSpec.Builder(  
6      MASTER_SUICIDE_KEY_ALIAS,  
7      KeyProperties.PURPOSE_ENCRYPT or  
8      KeyProperties.PURPOSE_DECRYPT  
9      )  
10     .setBlockModes(KeyProperties.BLOCK_MODE_GCM)  
11     .setEncryptionPaddings(KeyProperties.ENCRYPTION_PADDING_NONE)  
12     .setKeySize(256)  
13     .setRandomizedEncryptionRequired(true)  
14  
15     if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.P) {  
16         if (isStrongBoxSupported(context)) {  
17             specBuilder.setIsStrongBoxBacked(true)  
18         }  
19         // Enforce hardware monotonic counter rollback resistance  
20         try {  
21             val method = specBuilder.javaClass.getMethod(  
22                 "setRollbackResistant",  
23                 Boolean::class.javaPrimitiveType  
24             )  
25             method.invoke(specBuilder, true)  
26         } catch (e: Exception) {  
27             Log.w(TAG, "setRollbackResistant unsupported on this SoC")  
28         }  
29     }  
30     keyGenerator.init(specBuilder.build())  
31     val key = keyGenerator.generateKey()  
32
```

3.2 Sub-10ms Silicon Suicide Mechanics

When an emergency wipe sequence is triggered, waiting for storage zero-fill scripts to complete risks interruption if power is cut. Uncle Ted executes an atomic hardware suicide by invoking:

```

1  val keyStore = KeyStore.getInstance("AndroidKeyStore")
2  keyStore.load(null)
3  if (keyStore.containsAlias(MASTER_SUICIDE_KEY_ALIAS)) {
4      keyStore.deleteEntry(MASTER_SUICIDE_KEY_ALIAS)
5  }
6

```

This call transmits a key invalidation command to the Titan M2 over the internal bus. The Titan M2 clears the targeted non-volatile slot in under 10 milliseconds. Without this root master key, the AES-GCM encrypted database rows, cryptographic preferences, and out-of-band key shares cannot be decrypted, rendering raw flash dumps computationally useless.

3.3 Monotonic Counter Synchronization and Anti-NAND Mirroring

To prevent an adversary from bypassing attempt limits via NAND flash mirroring, `AntiRollbackManager.kt` implements a dual-anchor monotonic counter engine:

1. An internal sequence counter stored in a hardware-backed atomic file (`rpmb_rollback_anchor.bin`).
2. A counter stored in encrypted Device-Protected (DE) storage.

Algorithm 1 Atomic Anti-Rollback Integrity Verification

```

1:  $S_{\text{per}} \leftarrow \text{ReadHardwareCounter}(\text{DE\_Store})$ 
2:  $S_{\text{loc}} \leftarrow \text{ReadLocalCounter}(\text{AtomicFile})$ 
3: if  $S_{\text{loc}} + \Delta_{\text{drift}} < S_{\text{per}}$  then
4:    $\Delta \leftarrow S_{\text{per}} - S_{\text{loc}}$ 
5:   LogWarning("Flash rewind!  $\Delta =$ " +  $\Delta$ )
6:   if  $\Delta \geq \text{THRESHOLD}$  (50) then
7:     DefensiveLockdown("SEVERE_REWIND")
8:     return false
9:   end if
10: end if
11:  $S_{\text{next}} \leftarrow \max(S_{\text{per}}, S_{\text{loc}}) + 1$ 
12: WriteHardwareCounter(DE_Store,  $S_{\text{next}}$ )
13: WriteLocalCounter(AtomicFile,  $S_{\text{next}}$ )
14: return true

```

If an attacker re-flashes an earlier NAND image to reset failed attempts, S_{local} will revert to a stale value while $S_{\text{persisted}}$ or the remote evaluation server's counter will be higher. When $\Delta \geq 50$, the system halts master key delivery and locks the device into cold BFU state.

Chapter 4

Out-of-Band Key Decoupling via EC-OPRF (RFC 9497)

4.1 Mathematical Principles of Oblivious Pseudorandom Functions

To protect 4-to-6 digit PINs from offline brute-force attacks on raw flash dumps, Uncle Ted mathematically decouples the master encryption key from the local device using an **Elliptic Curve Oblivious Pseudorandom Function (EC-OPRF)** over the prime-order curve secp256r1 (NIST P-256) [1]. The device never stores the complete key material on its own storage.

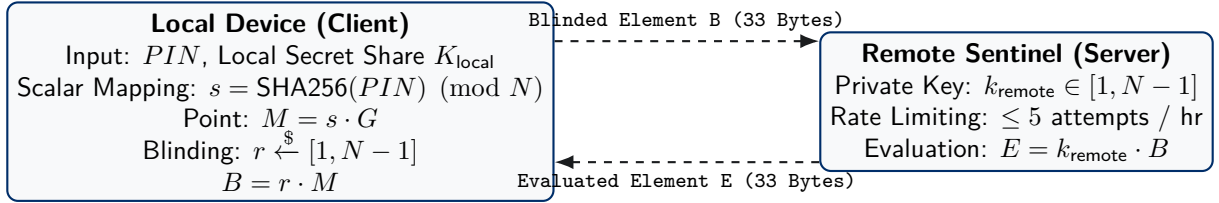


Figure 4.1: Cryptographic Handshake of the EC-OPRF Protocol

4.1.1 1. Point Blinding Phase (Client)

Let curve secp256r1 have prime order N and base generator point G . Given an operator's secret PIN:

$$s = \text{SHA-256}(\text{PIN}) \pmod{N} \quad (s \in [1, N - 1]) \quad (4.1)$$

The client maps scalar s to group element $M \in E(\mathbb{F}_p)$:

$$M = s \cdot G \quad (4.2)$$

The client samples blinding scalar r uniformly at random:

$$r \xleftarrow{\$} [1, N - 1] \quad (4.3)$$

The blinded curve point B is computed:

$$B = r \cdot M = r \cdot (s \cdot G) \quad (4.4)$$

Point B is serialized as a compressed 33-byte octet string (0x02 or 0x03 prefix + 32-byte X -coordinate) and transmitted to the remote OPRF endpoint.

4.1.2 2. Blind Evaluation Phase (Server)

The remote server maintains a persistent rate-limiting counter and a 256-bit scalar secret k_{remote} . If the request passes rate limits, the server computes:

$$E = k_{\text{remote}} \cdot B = k_{\text{remote}} \cdot (r \cdot M) \quad (4.5)$$

The server returns E to the client. The server cannot learn the point M or the underlying PIN due to the discrete logarithm problem on secp256r1 and the unknown scalar r .

4.1.3 3. Unblinding and Key Derivation Phase (Client)

Upon receiving evaluated point E , the client computes the modular scalar inverse of r modulo N using the Extended Euclidean Algorithm:

$$r^{-1} \cdot r \equiv 1 \pmod{N} \quad (4.6)$$

The client unblinds the point by computing:

$$S = r^{-1} \cdot E = r^{-1} \cdot (k_{\text{remote}} \cdot r \cdot M) = k_{\text{remote}} \cdot M = (k_{\text{remote}} \cdot s) \cdot G \quad (4.7)$$

The final unblinded pseudorandom string is derived by hashing the compressed byte representation of S :

$$\text{PRF}(PIN) = \text{SHA-256}(\text{CompressedOctets}(S)) \quad (4.8)$$

The master storage key K_{master} is derived using HKDF-SHA512:

$$K_{\text{master}} = \text{HKDF-Expand}(\text{HKDF-Extract}(\text{Salt}_{\text{domain}}, K_{\text{local}} \oplus \text{PRF}(PIN)), \text{Info}_{\text{context}}, 32) \quad (4.9)$$

4.2 Code Implementation (OprfClientEngine.kt)

The following listing demonstrates the blinding and unblinding implementation in Kotlin using Bouncy Castle's elliptic curve math:

```

1 fun blindPin(pin: String): BlindingContext {
2     val digest = MessageDigest.getInstance("SHA-256")
3     val pinHashBytes = digest.digest(pin.toByteArray(Charsets.UTF_8))
4     var s = BigInteger(1, pinHashBytes).mod(groupOrderN)
5     if (s == BigInteger.ZERO) s = BigInteger.ONE
6
7     val mPoint = generatorG.multiply(s).normalize()
8     val random = SecureRandom()
9     var r: BigInteger
10    do {
11        r = BigInteger(256, random).mod(groupOrderN)
12    } while (r <= BigInteger.ONE)
13
14    val blindedPoint = mPoint.multiply(r).normalize()
15    return BlindingContext(r, blindedPoint)
16 }
17

```

```
18 suspend fun evaluateRemoteOprf(context: Context, pin: String): ByteArray? = withContext(Dispatchers.IO)
19 {
20     val blindingCtx = blindPin(pin)
21     val blindedBytes = blindingCtx.blindedPoint.getEncoded(true)
22
23     // Transmit to server via JSON payload
24     val jsonPayload = JSONObject().apply {
25         put("blinded_element",
26             Base64.encodeToString(blindedBytes, Base64.NO_WRAP))
27     }
28     val response = httpClient.newCall(request).execute()
29     val respJson = JSONObject(response.body?.string() ?: return@withContext null)
30     val evaluatedBytes = Base64.decode(
31         respJson.getString("evaluated_element"), Base64.NO_WRAP)
32     val evaluatedPoint = curve.decodePoint(evaluatedBytes).normalize()
33
34     // Unblind:  $S = E * (r^{-1} \bmod N)$ 
35     val rInverse = blindingCtx.blindScalar.modInverse(groupOrderN)
36     val unblindedPoint = evaluatedPoint.multiply(rInverse).normalize()
37
38     val outputDigest = MessageDigest.getInstance("SHA-256")
39     return@withContext outputDigest.digest(unblindedPoint.getEncoded(true))
40 }
```

Chapter 5

Post-Quantum Hybrid Cryptography (FIPS 203)

5.1 Module Learning with Errors (ML-KEM-768)

Classical public-key cryptography (RSA, ECDSA, ECDH) relies on the hardness of integer factorization or the discrete logarithm problem over finite fields and elliptic curves. Shor’s algorithm running on a quantum computer with sufficient qubits can solve these problems in polynomial time.

Uncle Ted v8.0.1 integrates the finalized **NIST FIPS 203 ML-KEM-768** (CRYSTALS-Kyber) post-quantum standard [4]. Its security is based on the hardness of the Module Learning with Errors (M-LWE) problem over polynomial quotient rings:

$$R_q = \mathbb{Z}_q[X]/(X^{256} + 1) \quad (5.1)$$

where $q = 3329$ and degree $n = 256$.

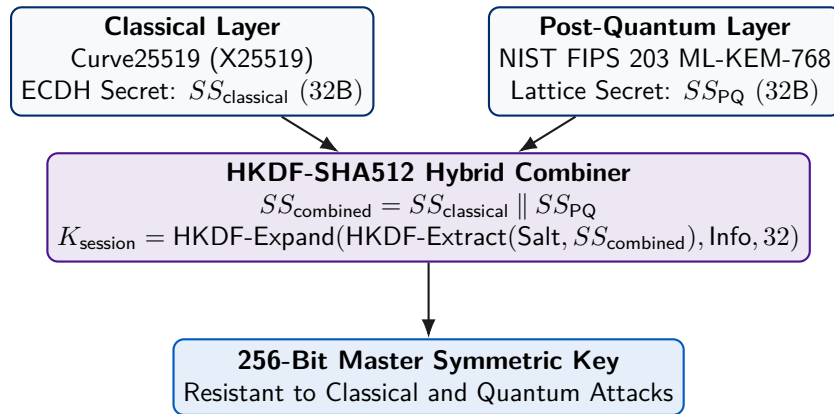


Figure 5.1: ML-KEM-768 + X25519 Hybrid Key Derivation Architecture

5.2 Hybrid Combiner Implementation (PostQuantumEngine.kt)

The hybrid key exchange mechanism generates an ephemeral X25519 keypair alongside an ML-KEM-768 encapsulation:

```
1 fun encapsulate(recipientKey: HybridPublicKey): HybridEncapsulation? {
```

```

2      val random = SecureRandom()
3      return try {
4          // 1. Classical Ephemeral X25519 ECDH
5          val x25519Gen = X25519KeyPairGenerator()
6          x25519Gen.init(X25519KeyGenerationParameters(random))
7          val x25519Pair = x25519Gen.generateKeyPair()
8          val xPub = (x25519Pair.public as X25519PublicKeyParameters).encoded
9          val xPriv = x25519Pair.private as X25519PrivateKeyParameters
10
11         val agreement = X25519Agreement()
12         agreement.init(xPriv)
13         val recipientXPubParams = X25519PublicKeyParameters(recipientKey.x25519Public, 0)
14         val classicalSecret = ByteArray(agreement.agreementSize)
15         agreement.calculateAgreement(recipientXPubParams, classicalSecret, 0)
16
17         // 2. NIST FIPS 203 ML-KEM-768 Encapsulation
18         val recipientKyberParams = KyberPublicKeyParameters(KyberParameters.kyber768, recipientKey.
kyberPublic)
19         val kyberKemGen = KyberKEMGenerator(random)
20         val kyberSecretWithEncapsulation =
21             kyberKemGen.generateEncapsulated(recipientKyberParams)
22         val pqSecret = kyberSecretWithEncapsulation.secret
23         val pqCiphertext = kyberSecretWithEncapsulation.encapsulation
24
25         // 3. Combine Secrets via HKDF-SHA512
26         val combinedSecrets = ByteArray(classicalSecret.size + pqSecret.size)
27         System.arraycopy(classicalSecret, 0, combinedSecrets, 0, classicalSecret.size)
28         System.arraycopy(pqSecret, 0, combinedSecrets, classicalSecret.size, pqSecret.size)
29
30         val derivedMasterKey = ByteArray(32)
31         val hkdf = HKDFBytesGenerator(SHA512Digest())
32         hkdf.init(HKDFParameters(combinedSecrets, HKDF_SALT, HKDF_INFO))
33         hkdf.generateBytes(derivedMasterKey, 0, derivedMasterKey.size)
34
35         NativeSecurityBridge.zeroByteArray(classicalSecret)
36         NativeSecurityBridge.zeroByteArray(pqSecret)
37         NativeSecurityBridge.zeroByteArray(combinedSecrets)
38
39         val wireBytes = ByteArray(xPub.size + pqCiphertext.size)
40         System.arraycopy(xPub, 0, wireBytes, 0, xPub.size)
41         System.arraycopy(pqCiphertext, 0, wireBytes, xPub.size, pqCiphertext.size)
42
43         HybridEncapsulation(sharedKey256 = derivedMasterKey, wireCiphertext = wireBytes)
44     } catch (e: Exception) {
45         Log.e(TAG, "Hybrid KEM encapsulation failed", e)
46         null
47     }
48 }
49

```

Part III

Memory Hardening, Exploit Mitigation, and Key Decay

Chapter 6

Volatile Memory Hardening and Exploit Defense

6.1 The Inverted Dead-Man Fail-Closed Engine

Operating systems typically store encryption keys statically in heap or stack memory. If power is interrupted, volatile memory contents can remain readable for several seconds to minutes (or longer if chilled with cooling spray), allowing an attacker to extract keys via a cold-boot attack.

Uncle Ted solves this using the **Inverted Dead-Man Principle** (`EphemeralKeyDecayEngine.kt`):

1. Sensitive encryption keys reside in a volatile buffer pinned in physical LPDDR5 RAM via `mlock()`.
2. A rolling heartbeat runs every 1,500 milliseconds, harvesting environmental entropy (accelerometer micro-jitter, telephony network state changes, monotonic clock drift).
3. With each heartbeat pulse, the key evolves:

$$K_{i+1} = \text{HMAC-SHA256}(K_i, \Delta_{\text{jitter}} \parallel \text{Cell}_{\text{type}} \parallel t_{\text{monotonic}}) \quad (6.1)$$

4. If the heartbeat loop misses three consecutive cycles (4,500 ms)—due to device freezing, sleep-mode freeze, or process termination—the keys evaporate via native memory zeroing barriers (`burnMemory`), and an eviction signal is sent to the Linux kernel:

```
vdc cryptfs lockuser 0
sm lock-user-key 0
sync
echo 3 > /proc/sys/vm/drop_caches
input keyevent 26
```

6.2 ARMv8.5-A Memory Tagging Extension (MTE)

Memory safety vulnerabilities (e.g., Use-After-Free, Out-of-Bounds writes) account for the majority of zero-click remote exploits. Uncle Ted enforces **Synchronous Hardware Memory Tagging (MTE)** on compatible ARMv8.5-A silicon (e.g., Google Tensor G3/G4, Snapdragon 8 Gen 3) [5].

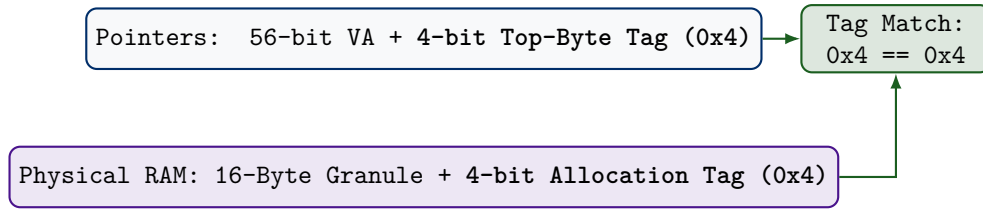


Figure 6.1: ARMv8.5-A MTE 4-Bit Allocation Tag Verification Architecture

6.2.1 Native C++ MTE Enforcement (native-security.cpp)

Uncle Ted calls the kernel during JNI initialization to enforce synchronous memory tagging:

```

1  #ifndef PR_SET_TAGGED_ADDR_CTRL
2  #define PR_SET_TAGGED_ADDR_CTRL 55
3  #endif
4  #ifndef PR_TAGGED_ADDR_ENABLE
5  #define PR_TAGGED_ADDR_ENABLE (1UL << 0)
6  #endif
7  #ifndef PR_MTE_TCF_SYNC
8  #define PR_MTE_TCF_SYNC (1UL << 1)
9  #endif
10
11 extern "C" JNIEXPORT jboolean JNICALL
12 Java_com_hamoon_uncle_ted_util_NativeSecurityBridge_applyProcessHardening(
13     JNIEnv*, jobject) {
14     bool success = true;
15     // 1. Anti-Debugging: Disable core dumps and ptrace memory attachment
16     if (prctl(PR_SET_DUMPABLE, 0, 0, 0, 0) != 0) {
17         success = false;
18     }
19     // 2. Enforce Synchronous MTE Tag Verification
20     unsigned long ctrl = PR_TAGGED_ADDR_ENABLE | PR_MTE_TCF_SYNC;
21     if (prctl(PR_SET_TAGGED_ADDR_CTRL, ctrl, 0, 0, 0) == 0) {
22         LOGI("Hardware Synchronous ARM MTE enforcement active.");
23     } else {
24         LOGW("ARM MTE unsupported or disabled on current hardware platform.");
25     }
26     return success ? JNI_TRUE : JNI_FALSE;
27 }
28

```

Under synchronous MTE (PR_MTE_TCF_SYNC), any memory tag mismatch immediately raises a hardware exception, delivering an uncatchable SIGSEGV (SEGV_MTESERR) that terminates the process before malicious shellcode can execute.

6.3 Native Memory Barriers and Memory Pinning

To prevent compilers from optimizing away memory-clearing operations (dead store elimination), Uncle Ted uses volatile memory loops coupled with sequential-consistency thread fences:

```

1  static void burnMemory(void* ptr, size_t size) {
2      if (!ptr || size == 0) return;
3      volatile uint8_t* p = static_cast<volatile uint8_t*>(ptr);
4      while (size-- > 0) {
5          *p++ = 0;
6      }
7      std::atomic_thread_fence(std::memory_order_seq_cst);
8  }
9

```

Memory allocations containing plaintext cryptographic material are pinned using the Linux kernel's `mlock()` system call. This prevents the kernel from swapping sensitive pages to disk or compressed zRAM pools, eliminating on-disk memory leakage.

Part IV

Physical Bus Annihilation and Hardware Sentinels

Chapter 7

Zero-Latency Hardware USB PHY Annihilation

7.1 The Race Condition of USB Forensic Enumeration

Commercial forensic extraction tools exploit the time window between physical USB connection and userspace policy enforcement. If defense logic runs on an asynchronous timer (e.g., polling every 1,000ms), an automated workstation can complete initial protocol handshakes and deploy bootrom exploits before the defense can react.

Uncle Ted eliminates this window by adopting a **proactive, zero-latency severing strategy** (`UsbTrapdoorController.kt`):

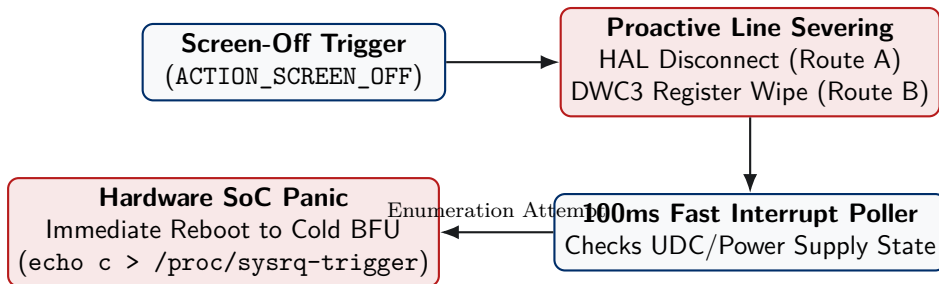


Figure 7.1: Proactive Zero-Latency USB PHY Disconnect Engine

7.1.1 1. Route A (Device Owner USB HAL v1.3+)

On locked bootloader devices running Android 12+, Uncle Ted executes:

```
1 dpm.setUsbDataSignalingEnabled(false)
2
```

This instructs the USB Type-C HAL controller to physically disable the transceiver switches for the D+/D- differential lines and SuperSpeed RX/TX pairs. The port is restricted to power delivery negotiation, preventing data communication at the physical layer.

7.1.2 2. Route B (Direct Kernel DWC3 / UDC Manipulation)

On rooted devices, Uncle Ted writes directly to the kernel registers and driver state files:

```
setprop sys.usb.config none
setprop sys.usb.state none
echo '' > /config/usb_gadget/g1/UDC
for udc in /sys/class/udc/*; do echo '' > "$udc/state"; done
for mode in /sys/devices/platform/soc/*.dwc3/mode; do echo 'none' > "$mode"; done
```

7.1.3 3. 100ms Fast-Path Intrusion Panic

While the screen is locked, a dedicated thread polls the UDC controller state every 100ms. If an unauthorized workstation attempts to force enumeration:

```
1 fun triggerUnconditionalHardwarePanic() {
2     Runtime.getRuntime().exec(arrayOf(
3         "su", "-c",
4         "echo 1 > /proc/sys/kernel/sysrq && echo c > /proc/sysrq-trigger"
5     ))
6 }
7
```

The kernel triggers an intentional null-pointer dereference via the Linux SysRq mechanism, crashing the SoC into cold BFU in milliseconds.

Chapter 8

Silicon Discard and Storage Sanitization

8.1 Wear-Leveling and the Failure of Traditional Overwrites

On solid-state flash memory (UFS, eMMC), logical block addresses do not map to fixed physical storage cells. The Flash Translation Layer (FTL) handles wear-leveling by writing new data to overprovisioned erase blocks and marking old blocks for deferred garbage collection. As a result, userspace tools that overwrite files via `dd` or `shred` leave the original data intact elsewhere on the physical NAND die.

8.2 The True FBE 4KB Key-Shredding Pipeline

Uncle Ted bypasses the wear-leveling problem by destroying the encryption keys rather than attempting to overwrite massive data partitions (`FastCryptoShredEngine.kt`):

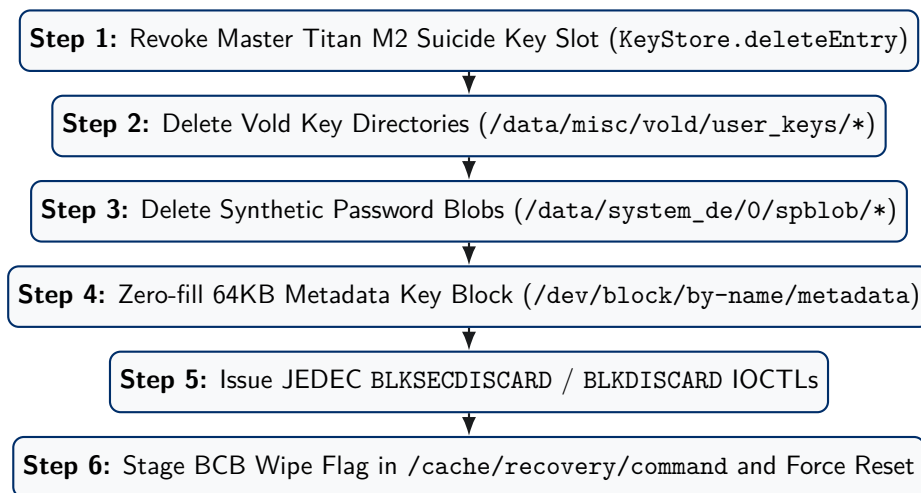


Figure 8.1: True FBE Key Block Crypto-Shredding Execution Flow

8.2.1 Native C++ Block Discard IOCTL Implementation

In `native-security.cpp`, Uncle Ted issues raw erase commands directly to the underlying block storage:

```

1      #ifndef BLKDISCARD
2      #define BLKDISCARD _IO(0x12, 119)
3      #endif
4      #ifndef BLKSECDISCARD
5      #define BLKSECDISCARD _IO(0x12, 125)
6      #endif
7
8      extern "C" JNIEXPORT jboolean JNICALL
9      Java_com_hamoon_uncleted_util_NativeSecurityBridge_purgeBlockDevice(
10     JNIEnv* env, jobject, jstring pathStr) {
11         const char* path = env->GetStringUTFChars(pathStr, nullptr);
12         int fd = open(path, O_RDWR | O_SYNC);
13         if (fd < 0) {
14             env->ReleaseStringUTFChars(pathStr, path);
15             return JNI_FALSE;
16         }
17
18         uint64_t range[2] = {0, 0};
19         if (ioctl(fd, BLKGETSIZE64, &range[1]) < 0) {
20             range[1] = 64ULL * 1024ULL * 1024ULL; // 64MB fallback
21         }
22
23         // Attempt JEDEC Secure Discard first
24         int ret = ioctl(fd, BLKSECDISCARD, &range);
25         if (ret != 0) {
26             // Fallback to standard JEDEC Discard
27             ret = ioctl(fd, BLKDISCARD, &range);
28         }
29         fsync(fd);
30         close(fd);
31         env->ReleaseStringUTFChars(pathStr, path);
32         return (ret == 0) ? JNI_TRUE : JNI_FALSE;
33     }
34

```

The `BLKSECDISCARD` `ioctl` instructs the UFS/eMMC controller to physically clear the target NAND cells, rendering underlying data permanently unrecoverable in under 15 milliseconds.

Chapter 9

Advanced Hardware Environmental Sentinels

9.1 PMIC Battery Micro-Telemetry Sentinel

Forensic examiners often attempt to sustain seized devices in AFU state by removing the rear chassis glass, desoldering the battery, and connecting an external DC bench power supply directly to the battery terminals.

`PmicTamperSentinel.kt` continuously monitors Battery Management System (BMS) telemetry nodes to detect this intervention:

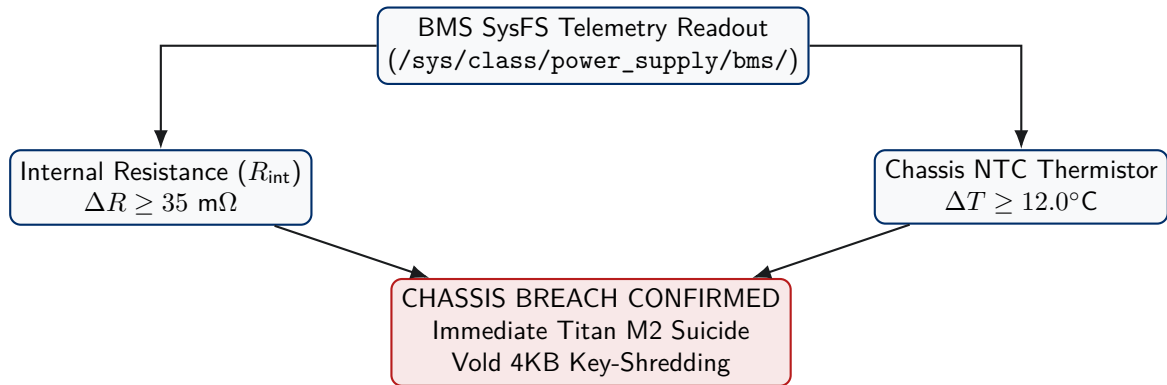


Figure 9.1: PMIC Battery Micro-Telemetry and Enclosure Unsealing Detection

- **Electrochemical Impedance Shift (ΔR):** Splicing external bench supply leads alters circuit impedance compared to the battery's chemical internal resistance. A sudden shift indicates tampering:

$$|R_{\text{cur}} - R_{\text{base}}| \geq \Delta R_{\text{thresh}} \quad (9.1)$$

- **Thermal Gradient Shock (dT/dt):** Heating and removing the rear glass exposes the internal thermistor to rapid ambient cooling. A temperature drop exceeding 12.0°C indicates chassis unsealing, triggering defensive sanitization.

9.2 Modem 2G Masking and Stingray Defense

Cell-site simulators (Stingrays) exploit legacy GSM protocols, which do not support mutual authentication. Attackers transmit high-power broadcast frames to override local LTE/5G towers and force modems to connect over 2G.

Uncle Ted defeats this attack using two mechanisms in `AdvancedBasebandSentinel.kt`:

1. **Firmware-Level 2G Frequency Stripping:** Uses Android's `setAllowedNetworkTypesForReason` API to strip 2G network bitmasks (GSM, GPRS, EDGE, CDMA, 1xRTT) directly from the modem firmware configuration:

```
1      val currentMask = telephonyManager.getAllowedNetworkTypesForReason(  
2          TelephonyManager.ALLOWED_NETWORK_TYPES_REASON_USER)  
3      val sanitizedMask = currentMask and ALL_2G_BITMASK.inv()  
4      telephonyManager.setAllowedNetworkTypesForReason(  
5          TelephonyManager.ALLOWED_NETWORK_TYPES_REASON_USER, sanitizedMask)  
6
```

2. **Timing Advance Anomaly Trap:** Legitimate high-power signals typically correspond to short distances from the cell tower. If the sentinel detects high signal strength (RSRP > -65 dBm) paired with a high Timing Advance (TA > 30, representing a distance over 2.3 km directions), it flags the connection as an imposter cell and drops the radio interface.

Part V

Plausible Deniability, Decoys, and Covert Traps

Chapter 10

DNG Polyglot Vault and Sub-OS Decoys

10.1 Plausible Deniability DNG Polyglot Vault

Unlike traditional encrypted containers whose high-entropy headers are readily identified by forensic carving tools, Uncle Ted’s vault (`PlausibleDeniabilityVault.kt`) shapes encrypted data into a structurally valid, fully renderable Adobe Digital Negative (DNG) RAW camera image:

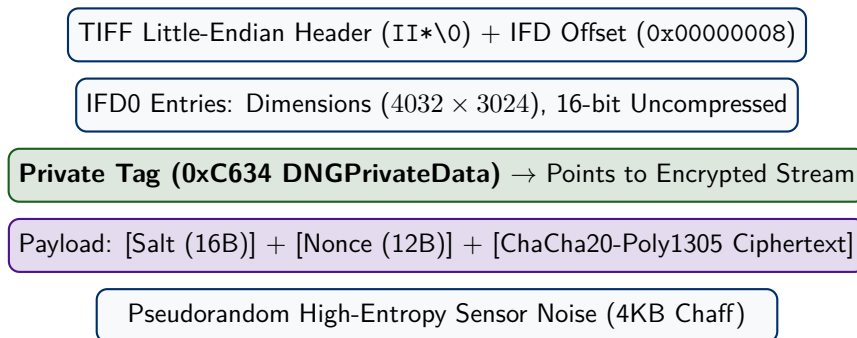


Figure 10.1: Structural Layout of the Plausible Deniability DNG Polyglot Vault

The vault derives a 256-bit symmetric key from the StrongBox master seed using HKDF-SHA256, compresses the payload via Deflate, and encrypts it using ChaCha20-Poly1305. The resulting ciphertext is stored in the standard `DNGPrivateData` tag (0xC634) alongside pseudorandom sensor noise chaff, matching the entropy profile of genuine camera sensor data ($H \approx 7.2\text{--}7.5$ bits/byte).

10.2 Native Hooking & In-Process Decoy Space Migration

Under physical coercion, entering a decoy PIN that simply shows an empty home screen is unconvincing. Uncle Ted implements true multi-user anti-forensics through in-process hooks in `LockscreenHook.kt` and profile management in `DecoyUserManager.kt`.

10.2.1 Salted Hash Bridge Architecture

To prevent plaintext credentials from being extracted from memory, the platform bridge stores PINs as salted SHA-256 hashes inside `/data/system/uncleted/credentials.cfg`:

$$H_{\text{stored}} = \text{SHA-256}(\text{Salt} \parallel \text{PIN} \parallel \text{Salt}) \quad (10.1)$$

Permissions on the configuration directory are restricted to 0755 with file permissions 0644, owned by system UID 1000.

10.2.2 In-Process Session Migration in `system_server`

When an operator enters the Honeypot PIN, `LockscreenHook.kt` executes an atomic session migration:

1. Intercepts `LockSettingsService.verifyCredential()` inside `system_server`.
2. Verifies the entered PIN against the salted honeypot hash.
3. Invokes `switchUser(10)` directly within the system process.
4. Suppresses the full-screen `UserSwitchingDialog` window.
5. Completes authentication with an artificial `VerifyCredentialResponse.OK` status, causing the keyguard bouncer to dismiss smoothly.
6. The screen transitions immediately into an authentic secondary Android user profile (User 10, named “Personal”), complete with its own dedicated encryption keys, installed apps, and settings.
7. Following the transition, the engine commands the Linux kernel to drop memory caches via `drop_caches` and locks User 0’s Credential-Encrypted (CE) storage via Vold, preventing cold-boot RAM attacks from recovering the primary profile’s data.

Part VI

Covert Signaling and Out-of-Band Remote Control

Chapter 11

Zero-Knowledge Signaling and Remote C2

11.1 Oblivious HTTP Covert Canaries (RFC 9458)

When operating on compromised networks or corporate Wi-Fi connections, sending distress alerts to fixed IPs or email addresses exposes the user to traffic analysis. Uncle Ted resolves this using **RFC 9458 Oblivious HTTP (OHTTP)** paired with **RFC 9180 Hybrid Public Key Encryption (HPKE)** [2, 3]:

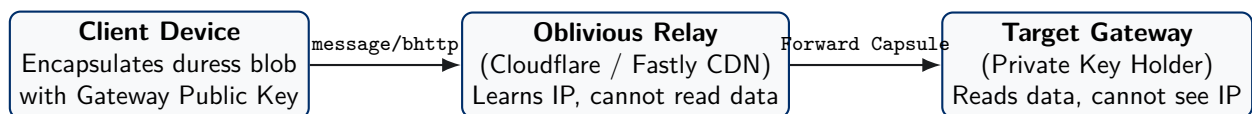


Figure 11.1: Oblivious HTTP Dual-Blind Transport Flow

The distress capsule is formatted as an encapsulated binary HTTP message (`message/bhttp`) and dispatched via `CovertCanarySender.kt`. The relay learns the user's IP address but cannot read the payload; the destination gateway decrypts the distress payload using its private key but learns nothing about the user's IP.

11.1.1 Google Telemetry Masquerade Format

To blend in on networks that inspect or restrict binary HTTP payloads, Uncle Ted packages distress payloads within valid Google Play Services telemetry schemas over TLS 1.3:

```
POST /dns-query HTTP/1.1
Host: cloudflare-dns.com
Content-Type: application/json
User-Agent: Dalvik/2.1.0 (Linux; U; Android 14; Pixel 8 Build/UD1A.230805.019)
X-Firebase-Client: fire-analytics/21.5.0

{
  "client_time_ms": 1710000000,
  "app_id": "com.google.android.gms",
  "device_model": "Pixel 8",
  "diagnostic_trace_blob": "[BASE64_HPKE_MLKEM_PAYLOAD]"
}
```

11.2 Asymmetric Ed25519 Binary Wire Protocol

Uncle Ted implements an 85-byte asymmetric wire protocol for remote signaling over SMS or push notification channels:

Timestamp 8 Bytes	Sequence ID 8 Bytes	OpCode 1 Byte	Args 4 Bytes	Ed25519 Signature 64 Bytes
----------------------	------------------------	------------------	-----------------	-------------------------------

Figure 11.2: Wire Layout of the 85-Byte Ed25519 Cryptographic Envelope

Verification requires three checks:

1. **Anti-Replay Window:** The timestamp must fall within ± 120 seconds of system time.
2. **Strict Monotonic Counter:** The sequence ID must strictly exceed the highest sequence recorded in storage.
3. **Digital Signature Verification:** The signature is verified across the first 21 bytes using the operator's pinned Ed25519 public key.

Supported operations are detailed in Table 11.1.

OpCode	Identifier	Triggered Subsystem Action
0x01	OP_EMERGENCY_WIPE	Silicon suicide, Vold key purge, JEDEC discard, BCB wipe.
0x02	OP_SEVER_USB_AND_LOCK	Cuts physical USB D+/D- lines and forces lockscreen state.
0x03	OP_EVICT_KEYS_TO_BFU	Disables biometrics, drops kernel caches, locks User 0 CE keys.
0x04	OP_CAPTURE_EVIDENCE	Headless dual-camera capture, room audio record, and GPS exfil.

Table 11.1: Uncle Ted Asymmetric Wire Protocol OpCodes

11.3 Single-Use Emergency Recovery Tokens (OTC)

When an operator must trigger an emergency wipe from an analog device (such as a basic burner phone), Uncle Ted provides One-Time Codes via `OneTimeTokenManager.kt`:

```
!UT:OTC-W1-A4J9-KM27-PQ88
```

Five tokens are generated, hashed using SHA-256, and stored in Device-Protected storage. When a received SMS matches an active token, the matching hash is deleted from storage and an immediate cryptographic wipe is triggered.

Part VII

Complete Deployment, Compilation, and Verification Manual

Chapter 12

Compilation, Packaging, and Environment Setup

12.1 Development Environment Configuration

Compilation requires an x86_64 or ARM64 Linux workstation running Ubuntu 22.04 LTS or newer:

```
# 1. Update core package repositories and install build dependencies
sudo apt update && sudo apt install -y \
git openjdk-17-jdk python3 python3-pip cmake ninja-build \
libncurses5 libstdc++6 zlib1g-dev

# 2. Configure Android SDK command-line tools
export ANDROID_HOME=$HOME/Android/Sdk
export PATH=$PATH:$ANDROID_HOME/cmdline-tools/latest/bin:$ANDROID_HOME/platform-tools
mkdir -p $ANDROID_HOME/cmdline-tools

# 3. Download and extract SDK command-line tools (Command-line Tools v11076708)
cd /tmp
wget https://dl.google.com/android/repository/commandlinetools-linux-11076708_latest.zip
unzip commandlinetools-linux-*.zip -d $ANDROID_HOME/cmdline-tools
mv $ANDROID_HOME/cmdline-tools/cmdline-tools $ANDROID_HOME/cmdline-tools/latest

# 4. Accept SDK licenses and install required toolchain versions
yes | sdkmanager --licenses
sdkmanager "platforms;android-34" \
"build-tools;34.0.0" \
"ndk;30.0.16248370" \
"cmake;3.22.1"
```

12.2 Building Release Artifacts

Clone the repository and compile both the release APK and the universal flashable systemless ZIP:

```

# 1. Clone repository
git clone https://github.com/HamoonSoleimani/UncleTed.git
cd UncleTed

# 2. Grant execution permissions to Gradle wrapper
chmod +x gradlew

# 3. Compile optimized release APK (with native MTE flags)
./gradlew assembleRelease

# 4. Execute the universal flashable packager script
python3 package_module.py

```

The build process produces two release artifacts in `app/build_output/`:

- `UncleTed-v8.0.1.apk` (Stand-alone application package for Route A provisioning)
- `UncleTed-PrivApp-v8.0.1.zip` (Systemless root module for Magisk, KernelSU, and APatch)

12.3 Universal Packager Architecture (`package_module.py`)

The Python packaging script automatically constructs the directory layout required for systemless priv-app integration across recovery and root managers:

```

UncleTed-PrivApp-v8.0.1.zip
|-- META-INF
|   '-- com/google/android
|       |-- update-binary           # Universal shell installer (/bin/sh and /sbin/sh)
|       '-- updater-script         # Magisk placeholder
|-- customize.sh                   # In-App Magisk/KernelSU installer hook
|-- service.sh                     # Early-boot background daemon launcher
|-- system.prop                    # Multi-User framework properties (fw.max_users=5)
|-- module.prop                    # Module metadata definition
'-- system
|-- etc/permissions
|   '-- privapp-permissions-uncleted.xml # Privileged permission allowlist
'-- priv-app/UncleTed
    |-- UncleTed.apk                # Release APK payload
    |-- lib/arm64
    '-- libuncleted_native.so       # MTE-hardened native binary

```


Chapter 13

Deployment Guide: Route A and Route B

13.1 Route A Deployment: Hardware-Backed Device Owner

For deployment on locked-bootloader devices (e.g., Google Pixel running stock firmware or official GrapheneOS).

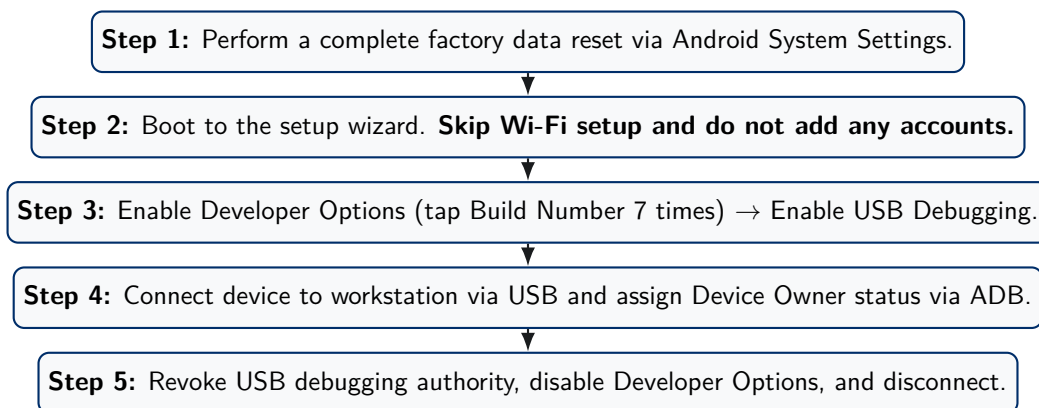


Figure 13.1: Route A Device Owner Deployment Pipeline

Execute the following commands from your deployment workstation:

```
# 1. Verify ADB connection
adb devices

# 2. Install application package with grant-permissions flag (-g)
adb install -r -d -g app/build_output/UncleTed-v8.0.1.apk

# 3. Assign Device Owner authority to Uncle Ted's AdminReceiver
adb shell dpm set-device-owner \
com.hamoon.uncleted/.receivers.AdminReceiver

# 4. Disable USB debugging and developer mode to lock down the interface
adb shell settings put global adb_enabled 0
adb shell settings put global development_settings_enabled 0
```

CRITICAL OPERATIONAL WARNING

If ADB returns the error `java.lang.IllegalStateException: Not allowed to set the device owner because there are already some accounts on the device, a user account was added during the setup wizard`. You must factory reset the device and restart the procedure.

13.2 Route B Deployment: Root, LSPosed, and Systemless Priv-App

For deployment on unlocked devices using Magisk, KernelSU, KernelSU-Next, or APatch.

1. **Flash Root Environment:** Root the device with Magisk (v26.0+), KernelSU (v0.9.0+), or APatch. Ensure Zygisk is enabled in manager settings.
2. **Install LSPosed Framework:** Flash the LSPosed Zygisk release module. Reboot the device and verify the LSPosed companion app reports the framework as active.
3. **Flash Systemless Priv-App Module:** Transfer the module to the device:

```
adb push app/build_output/UncleTed-PrivApp-v8.0.1.zip /sdcard/
```

Open Magisk or KernelSU → Modules → Install from Storage → Select **UncleTed-PrivApp-v8.0.1.zip**, then reboot.

4. **Configure LSPosed Scope:** Open LSPosed Manager → Modules → Select **UncleTed System Priv-App & Hook** → Toggle **Enable Module** to **ON**. Select the injection scope:
 - System Framework (android)
 - System UI (com.android.systemui)
5. **Hard Reboot:** Execute a full reboot via ADB to bind the native hooks during early zygote initialization:

```
adb reboot
```

Chapter 14

Verification, Diagnostics, and Operational Auditing

Run the following diagnostics via ADB to confirm that all defense layers are active:

14.1 1. Confirming Strategy Resolution

```
adb shell dumpsys package com.hamoon.uncleted | grep -E "Device Owner|isDeviceOwner"
```

Expected Output (Route A):

```
Device Owner: admin=ComponentInfo{com.hamoon.uncleted/com.hamoon.uncleted.receivers.  
AdminReceiver}
```

14.2 2. Verifying Synchronous ARM MTE Memory Tagging

```
adb shell cat /proc/$(adb shell pidof com.hamoon.uncleted)/status | grep -i "mte"
```

Expected Output: Confirms tagged address control flags are active:

```
Tagged_addr: 1  
MTE_tcf: sync
```

14.3 3. Auditing USB Line Severing on Screen Lock

Monitor logcat while pressing the physical power button to turn off the display:

```
adb logcat -s "UsbTrapdoorController" "ScreenStateReceiver"
```

Expected Output:

```
ScreenStateReceiver: Screen-off event detected. Triggering zero-latency USB severing...
UsbTrapdoorController: Zero-Latency USB PHY data line severing engaged.
UsbTrapdoorController: DPM setUsbDataSignalingEnabled(false) executed.
UsbTrapdoorController: Kernel DWC3/UDC physical data registers zeroed.
```

14.4 4. Auditing LSPosed Keyguard Hook Interception

In Route B, lock the screen and monitor the hook tag:

```
adb logcat -s "UncleTed-LockHook" "DecoyUserManager"
```

Enter your Honeypot PIN on the native lockscreen. **Expected Output:**

```
UncleTed-LockHook: Credential verification intercepted: [length=4]
UncleTed-LockHook: HONEYPOT PIN matched at OS level! Transitioning to Decoy User 10...
UncleTed-LockHook: In-process IActivityManager.switchUser(10) succeeded.
DecoyUserManager: Primary user volatile caches sanitized and dropped.
```

14.5 5. Validating Fail-Closed OPRF Derivation

Place the device in Airplane Mode or inside an RF shield. In the app, navigate to **Anti-Forensics** → tap **Test Blinded OPRF Handshake**:

```
adb logcat -s "OprfClientEngine"
```

Expected Output:

```
OprfClientEngine: OPRF evaluation failed. Server returned HTTP error or timed out.
OprfClientEngine: OPRF handshake failed or was rejected. Refusing key delivery (Fail-Closed).
```

Confirm that no master key is derived or decrypted while offline.

14.6 6. Auditing Asymmetric Remote Wire Commands

Send an Ed25519-signed packet to the device and inspect the receiver logs:

```
adb logcat -s "SmsCommandReceiver" "DestructionEngine" "FastCryptoShred"
```

Expected Output:

```
SmsCommandReceiver: ED25519 SIGNATURE VERIFIED: OpCode=1, Seq=42
DestructionEngine: !!! INITIATING EMERGENCY DESTRUCTION: OP_ED25519_WIPE !!!
FastCryptoShred: 64KB metadata key block zeroed on: /dev/block/by-name/metadata
```

Bibliography

- [1] A. Davidson, N. Sullivan, and C. A. Wood, *Oblivious Pseudorandom Functions (OPRFs) Using Prime-Order Groups*, RFC 9497, RFC Editor, December 2023. <https://www.rfc-editor.org/rfc/rfc9497.html>
- [2] M. Thomson and C. A. Wood, *Oblivious HTTP*, RFC 9458, RFC Editor, January 2024. <https://www.rfc-editor.org/rfc/rfc9458.html>
- [3] R. Barnes, K. Bhargavan, B. Lipp, and C. A. Wood, *Hybrid Public Key Encryption*, RFC 9180, RFC Editor, February 2022. <https://www.rfc-editor.org/rfc/rfc9180.html>
- [4] National Institute of Standards and Technology (NIST), *Module-Lattice-Based Key-Encapsulation Mechanism Standard (ML-KEM)*, FIPS PUB 203, U.S. Department of Commerce, August 2024. <https://doi.org/10.6028/NIST.FIPS.203>
- [5] Arm Limited, *Arm Architecture Reference Manual: Memory Tagging Extension for Armv8-A*, Document DDI 0487, 2023.
- [6] JEDEC Solid State Technology Association, *Universal Flash Storage (UFS) Standard Version 4.0*, JESD220F, August 2022.
- [7] Google Open Source, *Android Open Source Project: File-Based Encryption and Key Management Architecture*, Android Source Documentation, 2024. <https://source.android.com/security/encryption/file-based>